KYNARA

Complete Testing Guide

Step-by-step verification of Kynara — identity, roles, policies (including argument-level Slack/Gmail rules and dynamic downgrade on untrusted input), the MCP Gateway, Okta agent sync, approvals, audit, and the SDK.

Kynara · v2.4 · June 2026 · 18 tests

CONTENTS

1 Register & sign in

Goal: establish a valid org identity — the foundation for every other test.

1. Open kynaraai.com and click **Start free**. Fill email, a 12+ char password, display name, and org name.
2. Click **Create account** and confirm you land on the Dashboard with your org name and the trial badge.

✓ **Pass:** Dashboard loads; org name matches; trial badge visible.

2 Register an agent

Goal: create the agent identity your SDK/tests will reference.

1. **Agents** → **+ New agent**. Set display name “CRM Test Agent”, supervision `human_supervised`, daily budget 100.
2. Open the agent; copy the **Agent ID** (UUID).
3. Open **Access Summary** — it shows “No scopes granted” (expected; Test 3 fixes this).

✓ **Pass:** Agent appears with a UUID; Access Summary is empty.

3 Create a role, grant scopes & assign

Goal: pass the RBAC gate. An agent can only act if a role grants a matching scope.

1. **Access Control** → **Roles** → **+ New Role**: name “CRM Reader”; add scope `crm:contacts.read`; save.
2. **Agents** → **[agent]** → **Roles** → **Add role**: pick your user + the “CRM Reader” role.
3. Open **Access Summary** — it now lists `crm:contacts.read`.

✓ **Pass:** Access Summary shows `crm:contacts.read`.

IMPORTANT Scope notation is matched literally — keep `crm:contacts.read` (colon) consistent across role, policy, simulator, and SDK. This is the #1 cause of an unexpected “no role grants scope” deny.

4 Create policies (allow / deny / approval)

Goal: exercise all three outcomes and priority ordering.

1. **Policies** → **+ New policy** — Allow CRM reads (business hours): effect `allow`, priority 100, action `crm:contacts.read`, condition `time_between 09:00-18:00`.
2. Require approval off-hours: effect `require_approval`, priority 200, condition `not(time_between 09:00-18:00)`.
3. Deny blocked countries: effect `deny`, priority 50, condition `in(ctx.context.ip_country, ["CN","KP","RU"])`.
4. Confirm all three appear sorted 50 → 100 → 200.

✓ **Pass:** Three policies with correct effect badges; geo-deny (priority 50) evaluates first.

5 Argument-level policies (Slack / Gmail)

Goal: enforce intent on tool arguments, not just the action — channel allowlists and recipient-domain rules using `ctx.resource.attrs.*` and the `ends_with` operator.

1. Create a policy with action `slack.message.send` and condition: `in(ctx.resource.attrs.channel, ["C0123","C0456"])`, effect `allow`. (Or click *Load example* → *Slack: channel allowlist*.)

2. Create a policy with action `email.send` and condition `ends_with(ctx.resource.attrs.recipient, "@yourcompany.com")`, effect `allow`.
3. Add an external-recipient rule: `not(ends_with(... "@yourcompany.com"))`, effect `require_approval`.

1. In the Simulator, set Action `email.send` and Context/resource attrs `{"recipient":"bob@yourcompany.com"}` → expect **allow**.
2. Change recipient to `x@gmail.com` → expect **require_approval**.

> ✓ **Pass:** Internal recipient/allowed channel → allow; external recipient → require_approval.

6 Dynamic downgrade on untrusted input (is_tainted)

Goal (new): verify the `is_tainted` operator auto-tightens permissions once an agent ingests untrusted input — OWASP AI Exchange 'harden based on risk elevation' + MITRE ATLAS AML.M0030.

Policy + simulator

1. **Policies** → **+ New policy**: action `email.send` (an egress action), condition `{"op":"is_tainted"}`, effect `deny`, low priority number so it wins. (Or *Load example* → *Block data egress after untrusted input*.)
2. In the Simulator, Action `email.send`, Context `{}` → expect **allow** (no taint present).
3. Set Context `{"taint": true}` → expect **deny**.
4. Set Context `{"trust_level": "untrusted"}` → expect **deny** (coarse trust level also taints).

Category-scoped variant

1. Change the condition to `{"op":"is_tainted","args":["untrusted_web"]}`.
2. Context `{"taint":["untrusted_web"]}` → **deny**; Context `{"taint":["external_email"]}` → **allow** (different category).
3. Context `{"taint": true}` → **deny** (generic taint fail-safes to match any category).

MCP Gateway auto-taint (end-to-end)

1. Through the Kynara MCP wrapper, call a tool that returns untrusted content (e.g. `web_fetch` / `read_email`) → it succeeds and the wrapper logs `session.tainted`.
2. Then call an egress tool (e.g. `send_email`) in the same session → the `is_tainted` policy blocks it automatically.
3. Confirm the audit log shows the deny with the taint context.

> ✓ **Pass:** Clean request → allow; tainted request (flag, trust level, or after reading an untrusted source) → deny/approval; the gateway taints the session without agent code changes.

> TIP Use the softer template *Require approval for egress after untrusted input* (effect `require_approval`) when you want a human in the loop instead of an outright block.

7 Create an API key

Goal: credential for the SDK / decision endpoint.

1. **Settings** → **API Keys** → **+ New API Key**; scope `decisions.check`.
2. Copy the `sk_live_...` secret immediately (shown once).

> ✓ **Pass:** Key listed with prefix `sk_live_` and scope `decisions.check`.

8 Policy Simulator — all outcomes

Goal: verify the policy set before connecting a real agent (no quota used, no audit entry).

1. Open a policy → Simulator. Subject = your agent; action `crm:contacts.read`; context `{"time":"10:30","ip_country":"US"}` → **allow**.
2. Context `{"time":"22:30","ip_country":"US"}` → **require_approval**.
3. Context `{"time":"10:30","ip_country":"CN"}` → **deny**.
4. Action `payments.refund.issue` (no policy) → **deny** (default fail-closed).

> ✓ **Pass:** All four outcomes match; trace shows which policy matched.

> NOTE If allow returns “no role grants scope”, revisit Test 3 — the agent is missing its role assignment.

9 Approvals workflow

Goal: the human-in-the-loop lifecycle.

1. Run the off-hours simulation to create a pending approval.
2. **Approvals**: open the request, review context + risk score, approve with a note.
3. Run it again and reject a second request.
4. Check the Audit log for `approval.approved` and `approval.rejected`.

> ✓ **Pass:** Two resolved approvals; audit shows both events with your name + notes.

10 MCP Gateway — authorize tool calls

Goal: front an MCP server so every tool call is authorized per agent, with least-privilege discovery.

1. **Enforcement** → **MCP Gateway** → **Register server**: name, transport (sse), upstream URL, scope prefix (e.g. `mcp.crm`), fail mode `closed`.
2. Start the Kynara MCP wrapper with `KYNARA_MCP_SERVER_ID` + API key. On startup it discovers and syncs the upstream tools.
3. In the server detail, confirm tools appear; edit a tool's mapped scope/risk, or pin one to **deny**.
4. Use **Least-privilege preview**: enter the agent ID and confirm it only lists tools the agent may call (the denied tool is hidden).
5. From an MCP client pointed at the gateway, call an allowed tool (succeeds) and a denied tool (blocked with a Kynara error).

> ✓ **Pass:** Allowed tool returns a result; denied/hidden tool is blocked; decisions appear in the audit log.

11 Identity Providers — Okta agent sync

Goal: import agent identities from Okta and keep them in sync.

1. **Administration** → **Identity Providers** → **Connect Okta**: Okta org URL + SSWS token; sync mode **Agents** or **Group**.
2. Optionally add a role mapping (Okta group → Kynara role slug) and pick the on-behalf user.
3. Click **Test** (verifies connectivity), then **Sync now**.
4. Confirm imported agents appear under **Agents** with an Okta source; re-run sync and verify it updates (no duplicates).

✓ **Pass:** Test succeeds; agents import; re-sync updates rather than duplicates; mapped roles are granted.

12 Audit log integrity

Goal: confirm the tamper-evident chain and that prior tests are recorded.

1. **Audit log**: confirm recent events (login, agent.created, agent.assigned, decision.recorded, approvals, mcp/idp events).
2. Click **Verify Chain** → expect “Chain intact”.
3. Filter by event type / outcome; export CSV.

✓ **Pass:** “Chain intact — N events verified”; filters and CSV export work.

13 JIT grants (break-glass)

Goal: time-boxed elevation that auto-expires.

1. **Agents** → **[agent]** → **JIT Grants** → **New Grant**: scope `payments:refund.issue`, 5 min, a justification.
2. Confirm Access Summary shows the scope “via JIT grant” with an expiry; then Revoke and confirm it disappears.
3. Check the audit log for `jit_grant.created` / `jit_grant.revoked`.

✓ **Pass:** Scope appears then disappears on revoke/expiry; two audit entries.

14 Guardrails — auto-revocation

Goal: anomalous behavior auto-disables an agent.

1. **Guardrails**: create a rule — threshold 3 high-severity events in 5 min, action **revoke**.
2. Post 3 high-severity events for the agent; refresh the agent and confirm it shows as disabled.
3. Re-enable the agent.

✓ **Pass:** Agent is killed after the threshold; audit shows the guardrail trigger; re-enable works.

15 SDK — decision outcomes

Goal: verify the SDK end-to-end (allow / deny / require_approval) from code.

```
from kynara_sdk import Kynara
k = Kynara(api_key=KEY, agent_id=AGENT_ID, user_id=USER, fail_closed=True)
for ctx,expect in [({"time":"10:30","ip_country":"US"},"allow"),
                   ({"time":"22:30","ip_country":"US"},"require_approval"),
                   ({"time":"10:30","ip_country":"CN"},"deny")]:
    d = k.check(subject=("agent",AGENT_ID), action="crm:contacts.read",
                resource={"type":"crm.contact","id":"c1","attrs":{}}, context=ctx)
    assert d.effect == expect
```

✓ **Pass:** All three assertions pass; three new `decision.recorded` audit entries.

16 SDK — blocked before execution

Goal: prove the tool body never runs on a deny.

```
from kynara_sdk import permission_required, PermissionDenied
log=[]
@permission_required("crm.contacts.read", resource_arg="cid")
def read(cid):
    log.append(cid); return {"id":cid}
# allow context -> runs once; deny context (e.g. ip_country=CN) -> raises before body
assert len(log) == 1
```

✓ **Pass:** Denied call raises `PermissionDenied`; the function body ran only for the allowed call.

17 Human-in-the-loop from agent code

Goal: the full pause/approve/resume loop from code.

1. Call an action that returns `require_approval`; capture the `approval_id` and review URL.
2. Approve it in the Kynara UI.
3. Poll the approval via the API and confirm `status == approved`.

✓ **Pass:** Code reports PAUSED with an approval URL; after approval, status is `approved`.

18 MCP wrapper enforcement (end-to-end)

Goal: validate the gateway against a real MCP server, no agent changes.

1. Run the demo MCP server and the Kynara wrapper (with `KYNARA_MCP_SERVER_ID` + API key) per the wrapper README.
2. List tools as an agent: confirm only allowed tools are advertised (least-privilege discovery).
3. Call an allowed tool (forwarded, returns a result) and a denied tool (structured Kynara error).
4. Verify per-call decisions land in the audit log.

✓ **Pass:** Allowed tools forward; denied/hidden tools are blocked; every call is recorded.